Bézier Curve — Matrix Method

$$P(u) = U \cdot M \cdot G$$

Code Explanation for Julia Beginners

`03-bezier-matrix-pedagogical.jl`

E.R. Nurwijayadi

2026-03-15

Directed and curated by E.R. Nurwijayadi. Code and explanations AI-generated.

This document explains the Julia code in `03-bezier-matrix-pedagogical.jl` for readers new to Julia. The script uses no external libraries — only Julia's built-in matrix arithmetic. New concepts covered: matrix types, index arithmetic in `derive_M`, `reshape`, batch matrix evaluation, keyword arguments, `Union` types, `nothing`, and a general-purpose table printer with two display styles.

Contents

1 Type Map: How the Pieces Fit Together **3**

2 The `BezierMatrix` Struct **4**
2.1 `Matrix{Float64}` 4
2.2 The constructor 4

3 `derive_M` — Building the Basis Matrix **5**
3.1 What the formula computes 5
3.2 Index translation: Julia is 1-indexed 5
3.3 `zeros(Float64, n+1, n+1)` 6

4 `U_vec` and `evaluate_all` **6**
4.1 `U_vec` — building a single parameter row vector 6
4.2 `evaluate_all` — batch evaluation 7

5 Formatting Helpers **8**

5.1 `fmt` — integer or 4 decimal places 8
5.2 `fmt_frac` — float to exact fraction string 8
5.3 `fmt_poly` — coefficient vector to polynomial string 8

6 `print_table` — a General-Purpose Table Printer 9
6.1 Positional vs keyword arguments 9
6.2 `Union{Vector{String}, Nothing}` and `nothing` 10
6.3 `Symbol` and `:simple` / `:outline` 10
6.4 `something(label, "")` 10
6.5 Column width calculation 11
6.6 The two styles 11

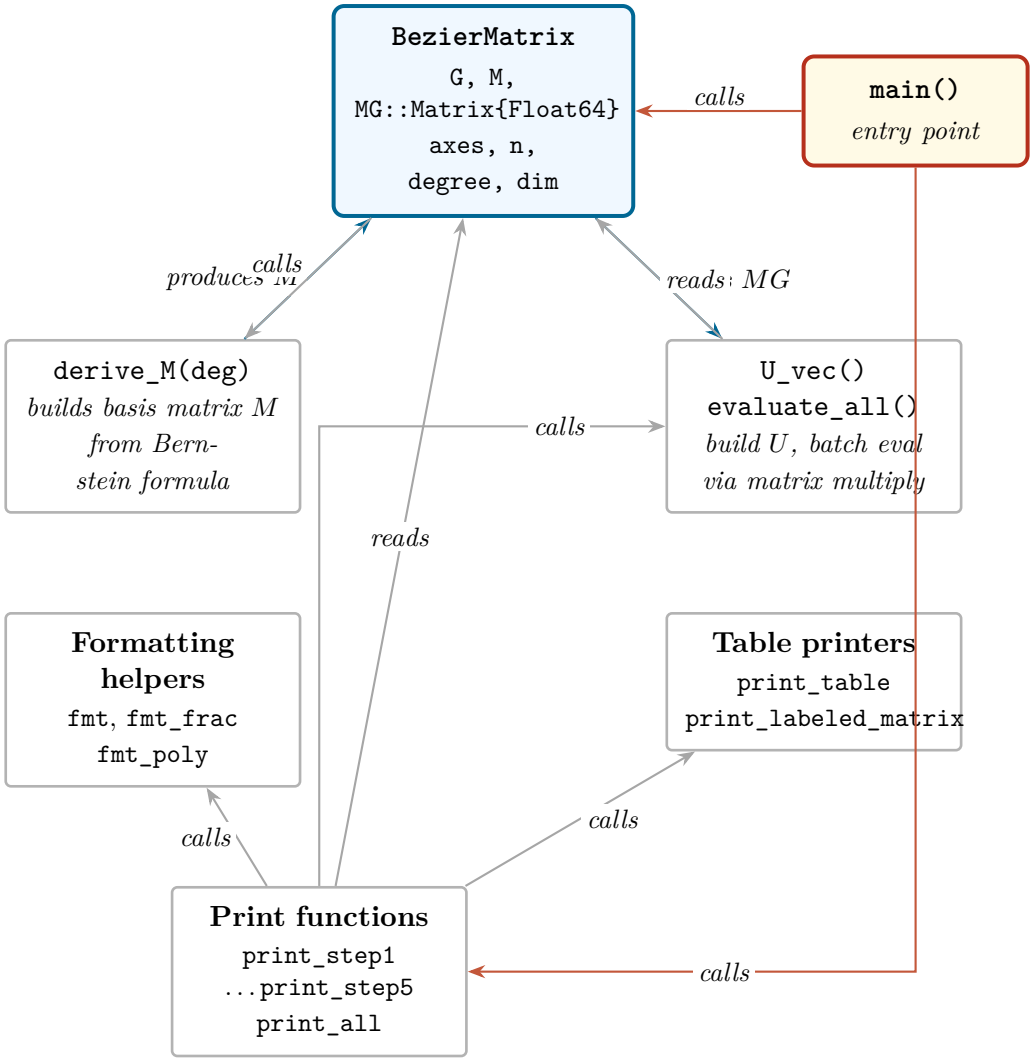
7 `print_labeled_matrix` 11

8 Print Functions 12
8.1 `print_step_header` — one-liner with parentheses 12
8.2 `print_step1_define` 13
8.3 `print_step2_derive_M` 13
8.4 `print_step3_MG` 15
8.5 `print_step4_evaluate` 16
8.6 `print_step5_table` 18

9 Entry Point and `print_all` 19

1. Type Map: How the Pieces Fit Together

*This script has no Poly type, no Symbolics.jl. Everything is plain matrix arithmetic. Two structs worth of logic are replaced by one struct and Julia's built-in * operator on matrices.*



What is genuinely new in this script. Scripts 01 and 02 had custom polynomial types and evaluation logic. This script has none of that — it uses Julia's built-in `Matrix{Float64}` and the `*` operator, which Julia already knows how to multiply. The novel parts are `derive_M` (the index arithmetic), `evaluate_all` (the 2-D array comprehension), and `print_table` (the general-purpose table formatter with keyword arguments and two display styles).

2. The BezierMatrix Struct

```julia
struct BezierMatrix
    G      ::Matrix{Float64}
    axes   ::Vector{String}
    n      ::Int
    degree ::Int
    dim    ::Int
    M      ::Matrix{Float64}
    MG     ::Matrix{Float64}
end
```

2.1. Matrix{Float64}

`Matrix{Float64}` is Julia's built-in 2-D array type for floating-point numbers. It is equivalent to `Array{Float64, 2}`. Julia stores it in column-major order internally, but you index it with `M[row, col]`.

The three matrix fields:

Field	Shape	Contents
`G`	$(n+1) \times \text{dim}$	The geometry matrix — control point coordinates. Row i = point P_{i-1}, column j = axis j.
`M`	$(n+1) \times (n+1)$	The basis matrix — Bernstein expansion coefficients. Column i = monomial coefficients of $B(n, i, u)$, top to bottom from highest power to constant.
`MG`	$(n+1) \times \text{dim}$	Pre-computed product $M \cdot G$. Row k = coefficient of u^{n-k} for each axis.

2.2. The constructor

```julia
function BezierMatrix(points::Matrix{Float64}, axes::Vector{String})
    n   = size(points, 1)
    deg = n - 1
    M   = derive_M(deg)
    BezierMatrix(points, axes, n, deg, length(axes), M, M * points)
end
```

`size(points, 1)` returns the number of rows in the matrix — the number of control points.

`M * points` is a plain Julia matrix multiplication. Because `M` is $(n+1) \times (n+1)$ and `points` is $(n+1) \times \text{dim}$, the result `MG` is $(n+1) \times \text{dim}$ — exactly what we need.

> For Case 2 (quartic, 3-D):
>
> `size(points, 1) = 5`, so `n = 5`, `deg = 4`.

> `M` is 5×5.
>
> `points` (i.e. `G`) is 5×3.
>
> `M * points` gives a 5×3 matrix `MG`.
>
> The final constructor call passes all seven fields in order: `points`, `axes`, `n=5`, `deg=4`, `length(axes)=3`, `M`, `M*points`.

3. `derive_M` — Building the Basis Matrix

This is the most intricate function in the script. Two nested loops, index arithmetic that needs careful reading, and a formula connecting Bernstein polynomials to matrix entries. Go slowly here.

```julia
function derive_M(deg::Int)::Matrix{Float64}
    n = deg
    M = zeros(Float64, n+1, n+1)
    for i in 0:n
        m = n - i
        for j in 0:m
            k = i + j
            M[n-k+1, i+1] = binomial(n,i) * binomial(m,j) * (-1)^j
        end
    end
    return M
end
```

3.1. What the formula computes

The entry $M[n-k+1,\ i+1]$ is the coefficient of u^{n-k} in the Bernstein polynomial $B(n,i,u)$.

This comes from expanding $B(n,i,u) = \binom{n}{i} u^i (1-u)^{n-i}$:

1. Apply the binomial theorem to $(1-u)^{n-i} = (1-u)^m$: each term contributes $\binom{m}{j}(-1)^j u^j$.
2. Multiplying by u^i shifts the power: $u^i \cdot u^j = u^{i+j}$.
3. So $k = i + j$ is the total power of u in that term.
4. Multiplying by $\binom{n}{i}$ gives the full coefficient.

3.2. Index translation: Julia is 1-indexed

The mathematical formula uses 0-based indices: column i (from 0 to n), row for power k (from 0 to n). Julia arrays are 1-indexed. The translation is:

- Column i in math $\to$ column `i+1` in Julia.
- Row for power u^k $\to$ highest power is at row 1, so power u^k is at row $n-k+1$.

Therefore `M[n-k+1, i+1]`.

Trace the loop for $n = 4$, $i = 1$, $j = 2$:

`m = n - i = 3`

`k = i + j = 1 + 2 = 3` (power of u is u^3)

`M[n-k+1, i+1] = M[4-3+1, 1+1] = M[2, 2]`

Value: $\binom{4}{1} \cdot \binom{3}{2} \cdot (-1)^2 = 4 \cdot 3 \cdot 1 = 12$

This says: the coefficient of u^3 in $B(4, 1, u)$ is $+12$.

Verify: $B(4, 1, u) = 4u(1-u)^3 = 4u - 12u^2 + 12u^3 - 4u^4$. The coefficient of u^3 is indeed $+12$. ✓

Why the inner loop runs `j in 0:m` and not `0:n`:

$B(n, i, u) = \binom{n}{i} u^i (1-u)^{n-i}$. The factor $(1-u)^m$ where $m = n - i$ contributes powers u^0 through u^m. After multiplying by u^i, the total power $k = i + j$ runs from i to $i + m = n$.

Entries for $k < i$ are zero (the polynomial has no u^k term below u^i). The loop simply never writes to those positions — they remain zero from the `zeros()` initialisation.

3.3. `zeros(Float64, n+1, n+1)`

`zeros(Float64, n+1, n+1)` creates an $(n+1) \times (n+1)$ matrix filled with `0.0`. This is important: entries that the loop never touches (where $k < i$, i.e. below the anti-diagonal) stay zero, which is the correct value — those Bernstein polynomial terms do not exist.

4. `U_vec` and `evaluate_all`

4.1. `U_vec` — building a single parameter row vector

```julia
function U_vec(deg::Int, u::Float64)::Matrix{Float64}
    reshape([u^(deg-k) for k in 0:deg], 1, deg+1)
end
```

The list comprehension `[u^(deg-k) for k in 0:deg]` produces a plain 1-D vector: $[u^{deg}, u^{deg-1}, \ldots, u^1, u^0]$.

`reshape(..., 1, deg+1)` changes its shape from a length-$(deg + 1)$ vector to a $1 \times (deg + 1)$ matrix — a single row.

This is necessary because Julia distinguishes between a length-5 vector (shape `(5,)`) and a 1×5 matrix (shape `(1,5)`). Matrix multiplication `U * MG` requires `U` to be a matrix, not a vector.

For `deg=4`, `u=0.5`:

`[0.5^4, 0.5^3, 0.5^2, 0.5^1, 0.5^0]` $= [0.0625, 0.125, 0.25, 0.5, 1.0]$

After `reshape`: a 1×5 matrix.

`U_vec(4, 0.5) * bm.MG` then gives a 1×3 matrix — the x, y, z values at $u = 0.5$.

4.2. `evaluate_all` — batch evaluation

```
function evaluate_all(bm::BezierMatrix,
                      u_values::Vector{Float64})::Matrix{Float64}
    n    = bm.degree
    Uall = [u^(n-k) for u in u_values, k in 0:n]
    Uall * bm.MG
end
```

This evaluates the curve at all u values simultaneously with a single matrix multiplication.

The 2-D array comprehension:

`[u^(n-k) for u in u_values, k in 0:n]`

This is a **two-variable comprehension**. The result is a 2-D matrix where:

- Each **row** corresponds to one value in `u_values`.
- Each **column** corresponds to one value of `k` (i.e. one power of u, from u^n down to u^0).

For `u_values = [0.0, 0.25, 0.5, 0.75, 1.0]` and `n=4`:

$$U_{\text{all}} = \begin{bmatrix} 0^4 & 0^3 & 0^2 & 0 & 1 \\ 0.25^4 & 0.25^3 & 0.25^2 & 0.25 & 1 \\ 0.5^4 & 0.5^3 & 0.5^2 & 0.5 & 1 \\ 0.75^4 & 0.75^3 & 0.75^2 & 0.75 & 1 \\ 1^4 & 1^3 & 1^2 & 1 & 1 \end{bmatrix}$$

U_{all} is 5×5. `MG` is 5×3. The product is 5×3 — five points, three coordinates each. Row i of the result is the curve point at `u_values[i]`.

Why this is faster than calling `U_vec` five times.

Calling `U_vec` once per u value then multiplying would perform five separate 1×5 times 5×3 multiplications. `evaluate_all` stacks all five U rows into one matrix and does one 5×5 times 5×3 multiplication. For large N (many u values), this is dramatically more efficient because BLAS (Julia's matrix library) is highly optimised for large matrix multiplications.

5. Formatting Helpers

5.1. fmt — integer or 4 decimal places

```
fmt(v::Float64) =
    abs(v - round(v)) < 1e-9 ? string(Int(round(v))) : @sprintf("%.4f", v)
```

A one-line function using the ternary operator `? :`. If `v` is within 10^{-9} of a whole number, print it as an integer; otherwise print 4 decimal places.

> `fmt(13.0)` → `"13"`
> `fmt(-32.0)` → `"-32"`
> `fmt(2.8125)` → `"2.8125"` (4 d.p.)

5.2. fmt_frac — float to exact fraction string

```
function fmt_frac(v::Float64)
    r = rationalize(Int, v; tol=1e-9)
    abs(Float64(r) - v) < 1e-9 || return @sprintf("%.4f", v)
    isone(denominator(r)) ? string(numerator(r)) : string(r)
end
```

This converts a `Float64` to its nearest exact rational representation and formats it as a string.

`rationalize(Int, v; tol=1e-9)` finds the simplest fraction p/q such that $|p/q - v| < 10^{-9}$.

The short-circuit check: `abs(Float64(r) - v) < 1e-9 || return ...`

This reads: "if the condition is false, return the fallback." The `||` operator short-circuits — if the left side is `false`, the right side (`return`) executes. This is a compact Julia idiom for early exit on failure.

> `fmt_frac(0.0625)` → `rationalize` finds `1//16` → denominator is not 1 → `"1//16"`
>
> `fmt_frac(5.0)` → `rationalize` finds `5//1` → `isone(denominator) = true` → `"5"`
>
> `fmt_frac(1.050781)` → `rationalize` finds `269//256` → `"269//256"`

5.3. fmt_poly — coefficient vector to polynomial string

```
function fmt_poly(coeffs::Vector{Float64}, deg::Int)
    terms = Tuple{Float64,String}[]
    for (k, c) in enumerate(coeffs)
        k -= 1
        abs(c) < 1e-9 && continue
        power = deg - k
        u_s   = power == 0 ? "" : (power == 1 ? "u" :
                 "u$(get(SUPERSCRIPTS, power, "^$power"))")
        c_s   = fmt(abs(c))
        c_s   = (c_s == "1" && !isempty(u_s)) ? "" : c_s
```

```
        push!(terms, (c, c_s * u_s))
    end
    isempty(terms) && return "0"
    parts = String[]
    for (idx, (c, t)) in enumerate(terms)
        push!(parts, idx == 1 ? (c < 0 ? "-$t" : t) :
                                (c < 0 ? "- $t" : "+ $t"))
    end
    join(parts, "  ")
end
```

This takes a column of `MG` (one coefficient per power of u) and formats it as a human-readable polynomial string.

Key difference from script 01's `fmt_poly`: Script 01 stored coefficients from constant to highest power (index 1 = u^0). Here the vector stores from *highest* to lowest power (index 1 = u^{deg}), matching the row ordering of `MG`. So the power is `deg - k` (0-indexed), not `k` directly.

For the x column of `MG`: `[13.0, -32.0, 24.0, 0.0, 0.0]` with `deg=4`:

k (0-idx)	c	power = deg-k	u_s	term
0	13.0	4	"u^4"	"13u^4"
1	-32.0	3	"u^3"	"32u^3"
2	24.0	2	"u^2"	"24u^2"
3	0.0	—	skipped (< 1e-9)	—
4	0.0	—	skipped	—

Result: `"13u^4 - 32u^3 + 24u^2"`

6. `print_table` — a General-Purpose Table Printer

This is the most complex function in the script — not mathematically, but in terms of Julia language features. It introduces keyword arguments, `Union` types, `nothing`, and `something()`. These are common Julia patterns worth understanding well.

```
function print_table(
        rows       ::Vector{Vector{String}},
        headers    ::Vector{String};
        row_labels ::Union{Vector{String}, Nothing} = nothing,
        style      ::Symbol = :simple)
```

6.1. Positional vs keyword arguments

The semicolon `;` in the argument list separates **positional arguments** (before `;`) from **keyword arguments** (after `;`).

Positional arguments must be passed in order without names. Keyword arguments are passed by name and have default values, so they are optional.

Calling with positional arguments only:

```
print_table(rows, headers)
# Uses defaults: row_labels=nothing, style=:simple
```

Calling with keyword arguments:

```
print_table(rows, headers; style=:outline)
print_table(rows, headers; row_labels=["u^4","u^3",...], style=:simple)
```

6.2. Union{Vector{String}, Nothing} and nothing

`Union{A, B}` is Julia's way of saying: "this argument can be either type `A` or type `B`."

`Nothing` is the type of the special value `nothing`, which represents the absence of a value — similar to `null` in other languages.

`Union{Vector{String}, Nothing}` therefore means: "either a vector of strings, or nothing (absent)."

The default `= nothing` means: if the caller does not provide `row_labels`, it defaults to `nothing`.

Inside the function, `has_rl = row_labels !== nothing` checks whether row labels were actually provided.

With row labels (e.g. printing matrix M):

```
print_table(rows, headers; row_labels=["u^4","u^3","u^2","u","1"])
```

Each row gets a label prepended: `"u^4 1 -4 6 -4 1"`.

Without row labels (e.g. printing the U vector):

```
print_table(u_row, u_headers)
```

No label column; rows are printed without a prefix.

6.3. Symbol and :simple / :outline

`Symbol` is a Julia type for interned strings — lightweight identifiers that begin with `:`. `:simple` and `:outline` are two symbol values used here as an enumeration to select the table style.

Using symbols rather than plain strings (`"simple"`) is idiomatic Julia: symbols are guaranteed to be unique (two `:simple` symbols are always the same object), and they are more efficient to compare.

6.4. something(label, "")

Inside `fmt_row`:

```
has_rl && push!(parts, lpad(something(label, ""), rl_w))
```

`something(x, default)` returns `x` if `x` is not `nothing`, otherwise returns `default`. Here, if `label` is `nothing` (no label for this row), it uses an empty string `""` as a placeholder, keeping the column width consistent.

6.5. Column width calculation

```
widths = [length(h) for h in headers]
for row in rows
    for (c, cell) in enumerate(row)
        widths[c] = max(widths[c], length(cell))
    end
end
```

This calculates the minimum width needed for each column: start with the header width, then expand to fit the widest cell in each column. Every cell is then left-padded (`lpad`) to this width, ensuring columns align perfectly regardless of content.

6.6. The two styles

```
if style == :outline
    println("  +=" * "="^(inner_w+2) * "=+")
    println("  | " * lstrip(hdr) * " |")
    println("  |=" * "="^(inner_w+2) * "=|")
    for (r, row) in enumerate(rows)
        println("  | " * lstrip(fmt_row(...)) * " |")
    end
    println("  +=" * "="^(inner_w+2) * "=+")
else
    println(hdr)
    println(ubar)
    for (r, row) in enumerate(rows)
        println(fmt_row(...))
    end
end
```

`:simple` — plain header with a dashed underline. Used for all intermediate tables (the M matrix, U vector, etc.).

`:outline` — box border using Unicode box-drawing characters (`+`, `|`, `=`). Used only for the final summary table in Step 5, to visually distinguish it.

> **`lstrip(hdr)` inside the outline style.** `fmt_row` prepends two spaces for alignment. Inside the `:outline` style, those leading spaces would push content away from the box border. `lstrip` strips them so the content sits flush against the `|` prefix.

7. print_labeled_matrix

```
function print_labeled_matrix(
        mat        ::Matrix{Float64},
        row_labels ::Vector{String},
        col_labels ::Vector{String};
        fmt_fn     = fmt,
        style      ::Symbol = :simple)
    rows = [[fmt_fn(mat[r,c]) for c in 1:size(mat,2)]
```

```
            for r in 1:size(mat,1)]
    print_table(rows, col_labels; row_labels=row_labels, style=style)
end
```

This is a thin convenience wrapper over `print_table`. It accepts a raw `Matrix{Float64}` and converts it to the `Vector{Vector{String}}` format that `print_table` expects, by applying `fmt_fn` to every element.

The keyword argument `fmt_fn = fmt`: `fmt_fn` is a **function-valued argument** with a default value of `fmt`. Callers can pass a different formatting function — for example `fmt_frac` to show fractions instead of decimals — without changing any other behaviour. This is a clean way to parameterise formatting.

Used in two different ways in the script:

```
# Print M matrix with integer formatting (default fmt)
print_labeled_matrix(bm.M, row_labels, col_labels)

# Print MG matrix with integer formatting (default fmt)
print_labeled_matrix(bm.MG, row_labels, bm.axes)
```

If you wanted fractions in the output, you could call:

```
print_labeled_matrix(bm.MG, row_labels, bm.axes; fmt_fn=fmt_frac)
```

8. Print Functions

All five print functions follow the same structure: they receive `bm::BezierMatrix` (and for steps 4 and 5, the `u_values` vector), read fields from `bm`, call helpers, and write to the terminal. None return a value.

8.1. `print_step_header` — one-liner with parentheses

```
print_step_header(step::Int, title::String) = (
    println();
    println("  STEP $step - $title");
    println("  " * "-"^(WIDTH-2))
)
```

Input: a step number (`Int`) and a title string.

Output:

```
  STEP 2 - Derive Basis Matrix M from Bernstein Polynomials
  ----------------------------------------------------------------
```

Instead of `function ... end`, this uses the short assignment form `name(args) = body`. Multiple statements are grouped with parentheses and separated by semicolons — purely a style choice, identical in behaviour to the long form.

8.2. print_step1_define

```
function print_step1_define(bm::BezierMatrix)
    print_step_header(1, "Matrix Formulation:  P(u) = U * M * G")
    n = bm.degree
    println()
    println("  P(u)  =  U(u)  *  M  *  G")
    println()
    println("  U(u)  shape (1x$(n+1))  parameter vector  [u^$n, ..., u,
    ↪  1]")
    println("  M     shape ($(n+1)x$(n+1))  basis matrix ...")
    println("  G     shape ($(n+1)x$(bm.dim))  geometry matrix ...")
    println()
    print_labeled_matrix(bm.G, ["P$(i-1)" for i in 1:bm.n], bm.axes)
end
```

Input: `bm` — reads `bm.degree`, `bm.n`, `bm.dim`, `bm.G`, `bm.axes`.

How it works: Prints the formula and the shape of each matrix using string interpolation (`$(n+1)` inserts the value at print time). Then calls `print_labeled_matrix(bm.G, ...)` to display the geometry matrix with row labels $P_0 \ldots P_4$ and column labels from `bm.axes`.

Output (Case 2):

```
STEP 1 - Matrix Formulation:  P(u) = U * M * G
----------------------------------------------------------------

P(u)  =  U(u)  *  M  *  G

U(u)  shape (1x5)  parameter vector  [u^4, ..., u, 1]
M     shape (5x5)  basis matrix  (derived from Bernstein)
G     shape (5x3)  geometry matrix  (control points)

    x  y  z
    -  -  -
P0  0  0  1
P1  0  4  1
P2  4  0  1
P3  4  4  1
P4  5  4  1
```

The table is `bm.G` printed by `print_labeled_matrix`, with row labels `["P0","P1","P2","P3","P4"]` and column labels `["x","y","z"]`.

8.3. print_step2_derive_M

```
function print_step2_derive_M(bm::BezierMatrix)
    print_step_header(2, "Derive Basis Matrix M from Bernstein
    ↪  Polynomials")
    n   = bm.degree
    sup = SUPERSCRIPTS
    ...
    for i in 0:n
```

```
        m = n - i
        b   = binomial(n, i) == 1 ? "" : string(binomial(n, i))
        u_p = i == 0 ? "" : (i == 1 ? "u" : "u$(get(sup, i, "^$i"))")
        v_p = m == 0 ? "" : (m == 1 ? "(1-u)" :
        ↪  "(1-u)$(get(sup,m,"^$m"))")
        println("  Column $i  ->  B($n,$i,u)  =  $(b)$(u_p)$(v_p)")
        for k in 0:n
            c = bm.M[k+1, i+1]
            abs(c) < 1e-9 && continue
            power = n - k
            u_str = power == 0 ? "1" : (power == 1 ? "u" : "u^$power")
            println("    M[$k,$i]  =  $(lpad(Int(round(c)), 4))   (coeff
            ↪  of $u_str)")
        end
        println()
    end
    println("  Full M matrix:")
    println()
    print_labeled_matrix(bm.M, ["u^$(n-k)" for k in 0:n],
                               ["P$(i-1)" for i in 1:bm.n])
end
```

Input: `bm` — reads `bm.degree`, `bm.M`.

How it works: Prints the derivation formula, then loops `i in 0:n` (one iteration per column of M, i.e. per Bernstein basis polynomial).

The **outer loop** builds the symbolic label for each $B(n, i, u)$ using the same superscript-dictionary trick as the previous scripts.

The **inner loop** iterates `k in 0:n` over all rows. It reads `bm.M[k+1, i+1]` (1-indexed), skips near-zero entries with `abs(c) < 1e-9 && continue`, and prints each non-zero entry with its power label.

Finally, `print_labeled_matrix(bm.M, ...)` prints the complete M matrix with row labels `["u^4","u^3",...]` and column labels `["P0","P1",...]`.

Output (Case 2, columns 0 and 1 shown):

```
  Column 0  ->  B(4,0,u)  =  (1-u)^4
    M[0,0]  =     1   (coeff of u^4)
    M[1,0]  =    -4   (coeff of u^3)
    M[2,0]  =     6   (coeff of u^2)
    M[3,0]  =    -4   (coeff of u)
    M[4,0]  =     1   (coeff of 1)

  Column 1  ->  B(4,1,u)  =  4u(1-u)^3
    M[0,1]  =    -4   (coeff of u^4)
    M[1,1]  =    12   (coeff of u^3)
    M[2,1]  =   -12   (coeff of u^2)
    M[3,1]  =     4   (coeff of u)

  Full M matrix:
```

```
      P0   P1   P2  P3  P4
      --  ---  ---  --  --
u^4    1   -4    6  -4   1
u^3   -4   12  -12   4   0
u^2    6  -12    6   0   0
u^1   -4    4    0   0   0
u^0    1    0    0   0   0
```

The inner loop only prints non-zero entries. Column 2 ($B(4,2,u) = 6u^2(1-u)^2$) starts at u^2, so rows for u^1 and u^0 are skipped entirely. Column 4 (u^4) has only one entry. The full matrix table at the end fills in the zeros that the per-column derivation omitted.

8.4. print_step3_MG

```julia
function print_step3_MG(bm::BezierMatrix)
    print_step_header(3, "Compute  M * G  - Polynomial Coefficient Matrix")
    n = bm.degree
    println()
    println("  M * G  gives the monomial coefficients for each axis.")
    println("  Row k of (M*G) is the coefficient of u^(n-k).")
    println()
    print_labeled_matrix(bm.MG, ["u^$(n-k)" for k in 0:n], bm.axes)
    println()
    println("  Resulting polynomials:")
    for (d, axis) in enumerate(bm.axes)
        println("    $(axis)(u)  =  $(fmt_poly(bm.MG[:,d], n))")
    end
end
```

Input: `bm` — reads `bm.degree`, `bm.MG`, `bm.axes`.

How it works: Prints the pre-computed `bm.MG` matrix as a labelled table, then loops over axes and calls `fmt_poly` on each column of `MG` to produce the polynomial string.

`bm.MG[:,d]` is Julia's column-slice syntax: all rows of column `d`. This gives a `Vector{Float64}` — the monomial coefficients for axis `d`, from highest power down.

Output (Case 2):

```
STEP 3 - Compute  M * G  - Polynomial Coefficient Matrix
----------------------------------------------------------------

M * G  gives the monomial coefficients for each axis.
Row k of (M*G) is the coefficient of u^(n-k).

       x    y  z
     ---  ---  -
u^4   13  -28  0
u^3  -32   64  0
u^2   24  -48  0
u^1    0   16  0
u^0    0    0  1
```

```
Resulting polynomials:
  x(u)  =  13u^4  - 32u^3  + 24u^2
  y(u)  =  -28u^4  + 64u^3  - 48u^2  + 16u
  z(u)  =  1
```

How `fmt_poly` reads the x column:

`bm.MG[:,1] = [13.0, -32.0, 24.0, 0.0, 0.0]` (index 1 = x axis, rows from u^4 down to u^0).

`fmt_poly` loops with 0-indexed `k`: `power = deg - k = 4 - k`.

k	coefficient	power	skipped?	term
0	+13	4	no	`"13u^4"`
1	−32	3	no	`"32u^3"`
2	+24	2	no	`"24u^2"`
3	0	1	yes (`< 1e-9`)	—
4	0	0	yes	—

Result: `"13u^4 - 32u^3 + 24u^2"`

8.5. print_step4_evaluate

```julia
function print_step4_evaluate(bm::BezierMatrix,
                              u_values::Vector{Float64})
    print_step_header(4, "Evaluate  U * (M * G)  at Each u Value")
    n = bm.degree
    for u in u_values
        Uv = U_vec(n, u)
        pt = vec(Uv * bm.MG)
        println()
        println("  -- u = $u " * "-"^(WIDTH - 10 - length(string(u))))
        u_headers = ["u^$(n-k)" for k in 0:n]
        u_row     = [[fmt_frac(Uv[1, k+1]) for k in 0:n]]
        println()
        println("  U  =")
        print_table(u_row, u_headers)
        println()
        println("  U * (M*G)  =  P(u):")
        for (d, axis) in enumerate(bm.axes)
            mg_col    = bm.MG[:, d]
            dot_parts = [
                "$(fmt_frac(Uv[1,k+1]))*($(fmt(mg_col[k+1])))"
                for k in 0:n
                if abs(mg_col[k+1]) > 1e-9 || abs(Uv[1,k+1]) > 1e-9
            ]
            println("    $(axis)(u)  =  $(join(dot_parts, "  +  "))")
            println("            =  $(fmt_frac(pt[d]))")
        end
    end
```

```
end
```

Input: `bm` (reads `bm.degree`, `bm.MG`, `bm.axes`) and `u_values::Vector{Float64}`.

How it works: Loops over each `u` value. For each one:

1. `U_vec(n, u)` builds the 1×5 row vector of powers.
2. `vec(Uv * bm.MG)` multiplies and flattens to a length-3 vector `pt` (one value per axis). `vec()` is needed because `Uv * bm.MG` returns a 1×3 matrix, not a vector.
3. The `U` table is printed with `print_table`, where `u_row` is wrapped in an extra `[...]` to make it a `Vector{Vector{String}}` — one row containing all the power values.
4. The dot product display loops over axes, reads the corresponding column of `bm.MG`, and builds the term strings. The `if` filter keeps a term if *either* $U[k]$ or $MG[k]$ is non-zero — so zero-coefficient terms are still shown when $U[k] \neq 0$, making the dot product fully explicit.

Output (Case 2, $u = 0.5$ block only):

```
-- u = 0.5 ------------------------------------------------

U  =
  u^4   u^3   u^2   u^1  u^0
-----  ----  ----  ----  ---
1//16  1//8  1//4  1//2    1

U * (M*G)  =  P(u):
  x(u)  =  1//16*(13)  +  1//8*(-32)  +  1//4*(24)
           +  1//2*(0)  +  1*(0)
         =  45//16
  y(u)  =  1//16*(-28)  +  1//8*(64)  +  1//4*(-48)
           +  1//2*(16)  +  1*(0)
         =  9//4
  z(u)  =  1//16*(0)  +  1//8*(0)  +  1//4*(0)
           +  1//2*(0)  +  1*(1)
         =  1
```

Tracing the x line for $u = 0.5$:

`Uv = [1/16, 1/8, 1/4, 1/2, 1]` (as `Float64`).

`mg_col = bm.MG[:,1] = [13, -32, 24, 0, 0]`.

The comprehension iterates `k = 0..4`:

k	Uv[1,k+1]	mg_col[k+1]	included?	term string
0	1/16	13	yes (both non-zero)	"1//16*(13)"
1	1/8	-32	yes	"1//8*(-32)"
2	1/4	24	yes	"1//4*(24)"
3	1/2	0	yes (U non-zero)	"1//2*(0)"
4	1	0	yes (U non-zero)	"1*(0)"

```
pt[1] = vec(Uv * bm.MG)[1] = 45/16 = 2.8125

fmt_frac(2.8125) → "45//16"
```

8.6. print_step5_table

```julia
function print_step5_table(bm::BezierMatrix,
                           u_values::Vector{Float64})
    print_step_header(5, "Summary - Evaluation Table")
    pts  = evaluate_all(bm, u_values)
    rows = Vector{Vector{String}}()
    for (i, u) in enumerate(u_values)
        row = [fmt(u)]
        for d in 1:bm.dim
            v = pts[i, d]
            push!(row, "$(fmt_frac(v))  ($(@sprintf("%.4f", v)))")
        end
        push!(rows, row)
    end
    println()
    print_table(rows, vcat(["u"], ["$(ax)(u)" for ax in bm.axes]);
                style=:outline)
end
```

Input: `bm` (reads `bm.dim`, `bm.axes`) and `u_values`.

How it works:

1. `evaluate_all(bm, u_values)` computes all points at once via the batch matrix multiplication, returning a 5×3 matrix `pts`.
2. Builds the `rows` vector by looping over `u_values`. For each row: the `u` value formatted with `fmt`, then for each axis `d`, the value `pts[i,d]` formatted as `"fraction (decimal)"` by combining `fmt_frac` and `@sprintf`.
3. `vcat(["u"], [...])` prepends `"u"` to the axis label list to form the full header row.
4. `print_table(...; style=:outline)` prints the box-bordered summary table.

Output (Case 2):

```
STEP 5 - Summary - Evaluation Table
-------------------------------------------------------------------


+=================================================================+
```

```
| u       x(u)                y(u)               z(u)        |
|=============================================================|
| 0       0  (0.0000)         0  (0.0000)   1  (1.0000)      |
| 0.2500  269//256  (1.0508)  121//64  (1.8906)  1  (1.0000) |
| 0.5000  45//16  (2.8125)    9//4  (2.2500)   1  (1.0000)   |
| 0.7500  1053//256  (4.1133) 201//64  (3.1406)  1  (1.0000) |
| 1       5  (5.0000)         4  (4.0000)   1  (1.0000)      |
+=============================================================+
```

Why Step 4 and Step 5 evaluate the curve differently:

Step 4 calls `U_vec` + `Uv * bm.MG` once per u value — one multiplication per point, useful for showing each individual dot product explicitly.

Step 5 calls `evaluate_all` once for all u values — one batch matrix multiplication, useful for efficiency and for gathering all results into a summary table.

Both routes produce the same numbers. The difference is pedagogical vs efficient.

9. Entry Point and `print_all`

```julia
function main()
    data = CASE_DATA[CASE]
    bm   = BezierMatrix(data.points, data.axes)
    print_all(bm, data.u_values, CASE)
end
```

`BezierMatrix(data.points, data.axes)` calls the constructor, which in turn calls `derive_M` and pre-computes `MG`. All heavy computation is done here, once.

`print_all` then calls the five print steps in order. Steps 1, 2, 3, and 5 read only from `bm` (no new computation). Step 4 builds `U_vec` per u value and multiplies — but this is cheap because `MG` is already available.

Computation summary:

When	What happens
Once, at construction	`derive_M`: double loop filling M `M * points`: one matrix multiply to get `MG`
Per u in Step 4	`U_vec`: one list comprehension + reshape `Uv * bm.MG`: one 1×5 times 5×3 multiply
Once in Step 5	`evaluate_all`: one 5×5 times 5×3 multiply